

Nome: Gustavo da Silva Pedroso Melo

Trabalho de LLP

Turma: EIEL21A

Data: 29/09/2026

1. Estrutura de dados é a maneira como organizamos as informações na memória para conseguir guardar, buscar e alterar elas com facilidade. Por exemplo, um programa de notas pode guardar as notas da turma em um vetor em vez de criar uma variável para cada aluno.
2. Vetor é um conjunto de valores do mesmo tipo com um único nome. Eles ficam um do lado do outro na memória e cada um é acessado pelo índice, que começa em 0.
3. `valores[0]` é 4 e `valores[3]` é 16. O número 20 está no índice 4, e o vetor tem 5 elementos.
4. O vetor é um bloco contínuo de memória com tamanho fixo. A lista encadeada é formada por nós espalhados, ligados uns aos outros, e pode crescer ou diminuir durante a execução.
5. Nó é cada elemento da lista. Ele guarda o dado (um número, por exemplo) e o endereço do próximo nó.
6. O ponteiro `proximo` indica onde está o nó seguinte. Sem ele os nós ficariam soltos e não teríamos como andar pela lista.
7. O último nó aponta para `nullptr` para mostrar que a lista acabou. É esse valor que faz o laço de percurso parar. Sem ele o programa tentaria ler um endereço inválido.
8. O vetor tem acesso direto a qualquer posição e gasta menos memória, já que não guarda ponteiros. A lista encadeada não precisa de tamanho definido antes e permite inserir e remover sem deslocar os outros elementos.

9.

```
#include <iostream>
```

```
using namespace std;
```

```
int main() {  
    int v[10];  
    for (int i = 0; i < 10; i++) {  
        cout << "Numero " << i + 1 << ": ";  
        cin >> v[i];  
    }  
    cout << "\nValores: ";  
    for (int i = 0; i < 10; i++) cout << v[i] << " ";  
    cout << endl;  
    return 0;  
}
```

10.

```
#include <iostream>  
using namespace std;
```

```
int main() {  
    int v[10];  
    for (int i = 0; i < 10; i++) {  
        cout << "Numero " << i + 1 << ": ";  
        cin >> v[i];  
    }  
    int iMaior = 0, iMenor = 0;  
    for (int i = 1; i < 10; i++) {  
        if (v[i] > v[iMaior]) iMaior = i;  
        if (v[i] < v[iMenor]) iMenor = i;  
    }
```

```

    }
    cout << "Maior: " << v[iMaior] << " (posicao " <<
iMaior << ")\n";
    cout << "Menor: " << v[iMenor] << " (posicao " <<
iMenor << ")\n";
    return 0;
}

```

11.

```

#include <iostream>
using namespace std;

int main() {
    double n[5], soma = 0;
    for (int i = 0; i < 5; i++) {
        cout << "Nota " << i + 1 << ": ";
        cin >> n[i];
        soma += n[i];
    }
    double media = soma / 5, maior = n[0], menor = n[0];
    int acima = 0;

    cout << "\nNotas:";
    for (int i = 0; i < 5; i++) {
        cout << " " << n[i];
        if (n[i] > maior) maior = n[i];
        if (n[i] < menor) menor = n[i];
        if (n[i] > media) acima++;
    }
}

```

```

    }
    cout << "\nMedia: " << media << "\nMaior: " << maior
        << "\nMenor: " << menor << "\nAcima da media: "
<< acima << endl;
    return 0;
}

```

12.

```

#include <iostream>
using namespace std;

```

```

int main() {
    int v[10], pares = 0;
    for (int i = 0; i < 10; i++) {
        cout << "Numero " << i + 1 << ": ";
        cin >> v[i];
        if (v[i] % 2 == 0) pares++;
    }
    cout << "Pares: " << pares << "\nImpares: " << 10 -
pares << endl;
    return 0;
}

```

13.

```

#include <iostream>
using namespace std;

```

```

int main() {
    int v[10], alvo, pos = -1;

```

```

for (int i = 0; i < 10; i++) {
    cout << "Numero " << i + 1 << ": ";
    cin >> v[i];
}
cout << "Procurar: ";
cin >> alvo;

for (int i = 0; i < 10 && pos == -1; i++)
    if (v[i] == alvo) pos = i;

if (pos != -1) cout << "Encontrado na posicao " << pos
<< endl;
else cout << "Valor não encontrado" << endl;
return 0;
}

```

14. Na lista $10 \rightarrow 25 \rightarrow 30 \rightarrow 45 \rightarrow \text{NULL}$, o primeiro elemento é o 10 e o último é o 45. Depois do 25 vem o 30. Há 4 nós, e o nó 45 aponta para NULL.

15. Ao inserir o 5 no início, o novo nó aponta para o 10 e vira a cabeça. Depois, ao inserir o 40 no final, o 30 passa a apontar para ele.

5 -> 10 -> 20 -> 30 -> NULL

5 -> 10 -> 20 -> 30 -> 40 -> NULL

16. Cada remoção só muda o ponteiro do nó anterior, ou a cabeça, no caso do primeiro.

Remover 5: 10 -> 20 -> 30 -> 40 -> NULL

Remover 30: 10 -> 20 -> 40 -> NULL

Remover 40: 10 -> 20 -> NULL

17.

O No*& é necessário porque a função altera o ponteiro início original. Com um ponteiro comum ela mudaria só uma cópia.

cpp

```
struct No {  
    int valor;  
    No* proximo;  
};
```

```
void inserirInicio(No*& inicio, int valor) {  
    No* novo = new No;  
    novo->valor = valor;  
    novo->proximo = inicio;  
    inicio = novo;  
}
```

18.

```
void exibir(No* inicio) {  
    for (No* p = inicio; p != nullptr; p = p->proximo)  
        cout << p->valor << " -> ";  
    cout << "NULL" << endl;
```

```
}
```

19.

```
bool buscar(No* inicio, int valor) {  
    for (No* p = inicio; p != nullptr; p = p->proximo)  
        if (p->valor == valor) return true;  
    return false;  
}
```

20.

```
#include <iostream>  
using namespace std;
```

```
const int N = 10;
```

```
double media(double n[]) {  
    double s = 0;  
    for (int i = 0; i < N; i++) s += n[i];  
    return s / N;  
}
```

```
int main() {  
    double n[N] = {0};  
    bool cadastrou = false;  
    int op;  
  
    do {
```

```

        cout << "\n1-Cadastrar 2-Exibir 3-Media
4-Maior/Menor "
            "5-Acima da media 6-Procurar 0-Sair\n> ";
    cin >> op;

    if (op >= 2 && op <= 6 && !cadastrou) {
        cout << "Cadastre as notas primeiro.\n";
        continue;
    }

    if (op == 1) {
        for (int i = 0; i < N; i++) {
            cout << "Nota do aluno " << i + 1 << ": ";
            cin >> n[i];
        }
        cadastrou = true;
    } else if (op == 2) {
        for (int i = 0; i < N; i++)
            cout << "Aluno " << i + 1 << ": " << n[i]
<< "\n";
    } else if (op == 3) {
        cout << "Media: " << media(n) << "\n";
    } else if (op == 4) {
        double mx = n[0], mn = n[0];
        for (int i = 1; i < N; i++) {
            if (n[i] > mx) mx = n[i];
            if (n[i] < mn) mn = n[i];
        }
    }

```

```

        cout << "Maior: " << mx << " | Menor: " << mn
<< "\n";
    } else if (op == 5) {
        double m = media(n);
        int c = 0;
        for (int i = 0; i < N; i++) if (n[i] > m) c++;
        cout << c << " aluno(s) acima da media\n";
    } else if (op == 6) {
        double x;
        bool achou = false;
        cout << "Nota procurada: ";
        cin >> x;
        for (int i = 0; i < N; i++)
            if (n[i] == x) {
                cout << "Aluno " << i + 1 << "\n";
                achou = true;
            }
        if (!achou) cout << "Nota não encontrada\n";
    } else if (op != 0) {
        cout << "Opcao invalida\n";
    }
} while (op != 0);

return 0;
}

```

21.

```
#include <iostream>
```

```
using namespace std;
```

```
struct No {  
    int valor;  
    No* proximo;  
};
```

```
// Insere no FINAL da lista
```

```
void inserir(No*& inicio, int valor) {
```

```
    No* novo = new No;  
    novo->valor = valor;  
    novo->proximo = nullptr;
```

```
    if (inicio == nullptr) {           // lista vazia  
        inicio = novo;
```

```
    } else {
```

```
        No* atual = inicio;
```

```
        while (atual->proximo != nullptr) // vai até o  
último
```

```
            atual = atual->proximo;
```

```
        atual->proximo = novo;
```

```
    }
```

```
}
```

```
void listar(No* inicio) {
```

```
    if (inicio == nullptr) {
```

```
        cout << "Lista vazia.\n";
```

```
        return;
```

```

    }
    for (No* atual = inicio; atual != nullptr; atual =
atual->proximo)
        cout << atual->valor << " -> ";
    cout << "NULL\n";
}

```

```

bool buscar(No* inicio, int valor) {
    for (No* atual = inicio; atual != nullptr; atual =
atual->proximo)
        if (atual->valor == valor) return true;
    return false;
}

```

```

bool remover(No*& inicio, int valor) {
    No* atual = inicio;
    No* anterior = nullptr;

    while (atual != nullptr && atual->valor != valor) {
        anterior = atual;
        atual = atual->proximo;
    }
    if (atual == nullptr) return false; // não achou

    if (anterior == nullptr)                // removendo o
primeiro
        inicio = atual->proximo;
    else
        anterior->proximo = atual->proximo;
}

```

```

        delete atual;                                // libera a
memória
        return true;
    }

int contar(No* inicio) {
    int total = 0;
    for (No* atual = inicio; atual != nullptr; atual =
atual->proximo)
        total++;
    return total;
}

void liberarLista(No*& inicio) {
    while (inicio != nullptr) {
        No* temp = inicio;
        inicio = inicio->proximo;
        delete temp;
    }
}

int main() {
    No* lista = nullptr;
    int opcao, valor;

    do {
        cout << "\n1 - Inserir elemento\n";

```

```
cout << "2 - Listar elementos\n";
cout << "3 - Buscar elemento\n";
cout << "4 - Remover elemento\n";
cout << "5 - Contar elementos\n";
cout << "0 - Sair\n";
cout << "Opcao: ";
cin >> opcao;

switch (opcao) {
    case 1:
        cout << "Valor a inserir: ";
        cin >> valor;
        inserir(lista, valor);
        break;
    case 2:
        listar(lista);
        break;
    case 3:
        cout << "Valor a buscar: ";
        cin >> valor;
        cout << (buscar(lista, valor) ?
"Encontrado!" : "Nao encontrado.") << endl;
        break;
    case 4:
        cout << "Valor a remover: ";
        cin >> valor;
        cout << (remover(lista, valor) ?
"Removido!" : "Valor nao existe na lista.") << endl;
```

```

        break;
    case 5:
        cout << "Total de elementos: " <<
contar(lista) << endl;
        break;
    case 0:
        cout << "Saindo...\n";
        break;
    default:
        cout << "Opcao invalida!\n";
    }
} while (opcao != 0);

liberarLista(lista);
return 0;
}

```

22. Para registros que entram e saem o tempo todo, a lista encadeada é mais adequada. No vetor, remover um item do meio obriga a deslocar todos os seguintes, e se o espaço acabar é preciso criar outro vetor maior e copiar tudo. Na lista, basta ajustar alguns ponteiros e o tamanho acompanha a necessidade. Em compensação, a lista não permite acesso direto: para chegar ao 500o registro é preciso passar pelos 499 anteriores, e cada nó gasta memória extra com o ponteiro. O vetor tem justamente a vantagem oposta, que é o acesso imediato pelo índice, mas sofre com o tamanho fixo e com o custo de inserir e remover. Se o sistema consultasse por posição o tempo todo e alterasse pouco, o vetor seria a melhor escolha.

Característica	Vetor	Lista Encadeada
Tamanho	Fixo, definido na criação	Varia durante a execução
Acesso aos elementos	Direto, pelo índice	Sequencial, a partir do início
Inserção/remoção	Lenta, desloca elementos	Rápida, só troca ponteiros
Uso de memória	Bloco contínuo, pode sobrar espaço	Só o necessário, mas com um ponteiro por nó
Exemplo de aplicação	Notas de uma turma fixa	Fila de atendimento

